

Resumen: Teoría de la Computación

IIC2213 – Lógica para Ciencia de la Computación

IIC2214 – Teoría de la Computación

Clases 6 a 12 – Preparación para Guías de Computabilidad y Lógica

Basado en el material del Prof. Miguel Romero – Pontificia Universidad Católica de Chile

Unidad I: Computabilidad

Clase 6 – Máquina Universal de Turing

Clase 7 – Problemas Indecidibles

Clase 8 – Reducciones e Indecidibilidad

Clase 9 – Más Indecidibilidad (PCP)

Unidad II: Lógica de Primer Orden

Clase 10 – LPO: Sintaxis y Semántica

Clase 11 – Definibilidad e Isomorfismos

Clase 12 – Definibilidad de Clases y Compacidad

Índice

I	Unidad I: Computabilidad	3
1.	Clase 6 – Máquina Universal de Turing (MUT)	3
1.1.	Máquina de Turing (MT): Recordatorio	3
1.2.	MTs codificables: Convenciones	3
1.3.	Codificación $\langle \cdot \rangle$ de una MT	3
1.4.	Máquina Universal de Turing (MUT)	4
2.	Clase 7 – Problemas Indecidibles	5
2.1.	Diagonal y el argumento de diagonalización	5
2.2.	Accept es indecidible	5
3.	Clase 8 – Reducciones e Indecidibilidad	6
3.1.	El Problema de la Parada (Halting)	6
3.2.	$\text{Halting}_\varepsilon$	6
3.3.	Reducciones Mucho a Uno ($\leq_m$)	6
3.4.	Problemas Clásicos	7
4.	Clase 9 – Más Indecidibilidad: PCP	8
4.1.	Problema de Correspondencia de Post (PCP)	8
4.2.	PCP Modificado (MPCP)	8
4.3.	$\text{ACCEPT} \leq_m \text{MPCP}$ (Construcción)	8
4.4.	$\text{MPCP} \leq_m \text{PCP}$	8
II	Unidad II: Lógica de Primer Orden (LPO)	9
5.	Clase 10 – LPO: Sintaxis y Semántica	9
5.1.	Vocabulario $\mathcal{L}$	9
5.2.	Sintaxis: $\mathcal{L}$ -términos	9
5.3.	Sintaxis: $\mathcal{L}$ -fórmulas	9
5.4.	Semántica: $\mathcal{L}$ -estructuras	9
5.5.	Variables libres y ligadas	9
5.6.	Asignaciones y satisfacción $\models$	10
5.7.	Equivalencia lógica	10
5.8.	Consecuencia lógica, satisfacibilidad, validez	10
5.9.	Problemas de decisión en LPO	11

6. Clase 11 – Definibilidad sobre una Estructura e Isomorfismos	12
6.1. Definibilidad sobre una estructura	12
6.2. Isomorfismos	12
6.3. Teorema de Isomorfismo	12
6.4. Isomorfismos y no definibilidad	13
7. Clase 12 – Definibilidad de Clases de Estructuras y Compacidad	14
7.1. Propiedades sobre $\mathcal{L}$ -estructuras	14
7.2. Propiedades cerradas bajo isomorfismos	14
7.3. Teorema de Compacidad	14
7.4. Estrategia de compacidad para no definibilidad	14
7.5. Ejemplos clásicos de no definibilidad por compacidad	14
7.5.1. P_{fin} = estructuras finitas	14
7.5.2. P_{path} = existe camino dirigido de c a d en $\mathcal{A}$	15
7.5.3. P_{cycle} = existe un ciclo dirigido finito	15
7.5.4. $P_{\text{fin-degree}}$ = existe elemento con grado de salida finito	15
III Herramientas y Técnicas para Ejercicios	16
8. Guía de Computabilidad – Técnicas Esenciales	16
8.1. Diseño de MTs (Problemas 1–3 de la guía)	16
8.2. Modelos extendidos de MT (Problemas 4–6)	16
8.3. Lenguajes decidibles y RE (Problemas 7–10)	16
8.4. Reducciones – Técnicas (Problemas 16–20)	16
9. Guía de Lógica – Técnicas Esenciales	18
9.1. Demostraciones con $\models$ y satisfacibilidad (Problemas 1–2)	18
9.2. Isomorfismos – Demostrar que dos estructuras NO son isomorfas (Problemas 3–6)	18
9.3. Definibilidad sobre una estructura (Problemas 7–14)	18
9.4. Definibilidad de propiedades/clases (Problemas 17–24)	18
IV Tablas Resumen	20
10. Jerarquía de Lenguajes en Computabilidad	20
11. Herramientas de LPO para Demostraciones	20

Parte I

Unidad I: Computabilidad

1. Clase 6 – Máquina Universal de Turing (MUT)

1.1. Máquina de Turing (MT): Recordatorio

Definición 1.1 (Máquina de Turing determinista). Una MT determinista es una tupla $M = (Q, \Sigma, \Gamma, q_0, q_f, \delta)$, donde:

- Q es un conjunto finito de estados.
- Σ es el alfabeto de entrada.
- Γ es el alfabeto de cinta, $\Sigma \subseteq \Gamma$, con $\vdash, B \in \Gamma \setminus \Sigma$.
- q_0 es el estado inicial.
- q_f es el estado final.
- $\delta : (Q \setminus \{q_f\}) \times \Gamma \rightarrow Q \times \Gamma \times \{\triangleleft, \triangleright\}$ es una función parcial de transición.

1.2. MTs codificables: Convenciones

Sin pérdida de generalidad, trabajamos con MTs que satisfacen:

- $\Sigma = \{0, 1\}$ y $\Gamma = \{0, 1, \vdash, B\}$.
- $Q = \{1, \dots, n\}$ con $n \geq 1$, $q_0 = 1$ y $q_f = n$.

Una MT que cumple estos supuestos se denomina **codificable**.

1.3. Codificación $\langle \cdot \rangle$ de una MT

Todo objeto discreto se representa como palabra en $\{0, 1\}^*$:

- **Estados:** $\langle i \rangle = 0^i$ (unario con 0).
- **Símbolos de cinta:** $\langle 0 \rangle = 0$, $\langle 1 \rangle = 00$, $\langle \vdash \rangle = 000$, $\langle B \rangle = 0000$.
- **Direcciones:** $\langle \triangleleft \rangle = 0$, $\langle \triangleright \rangle = 00$.
- **Transición** $\delta(q, a) = (q', b, D)$: $\langle t \rangle = \langle q \rangle 1 \langle a \rangle 1 \langle q' \rangle 1 \langle b \rangle 1 \langle D \rangle$.
- **Función** δ : $\langle \delta \rangle = \langle t_1 \rangle 11 \langle t_2 \rangle 11 \dots 11 \langle t_m \rangle$ (orden lexicográfico).
- **MT completa:** $\langle M \rangle = 0^n 111 \langle \delta \rangle$.
- **Par** (M, w) : $\langle M, w \rangle = \langle M \rangle 1111 w$.

Corolario 1.1. El conjunto $\mathcal{M}$ de todas las MTs codificables es **enumerable**. La función $\langle \cdot \rangle : \mathcal{M} \rightarrow \{0, 1\}^*$ es inyectiva.

Teorema 1.1 (Existencia de lenguajes indecidibles). La clase $\mathcal{R}_{\{0,1\}}$ de lenguajes decidibles sobre $\{0, 1\}$ es enumerable, pero la clase de todos los lenguajes sobre $\{0, 1\}$ es no enumerable. En consecuencia, **existen lenguajes indecidibles**.

1.4. Máquina Universal de Turing (MUT)

Definición 1.2 (MUT). *Una MUT U recibe $z \in \{0,1\}^*$, verifica que $z = \langle M, w \rangle$ para una MT M y palabra w , y luego ejecuta M sobre w .*

Construcción: 3 cintas:

1. Cinta 1: mantiene $\langle M, w \rangle$ (solo lectura).
2. Cinta 2: ejecución de M sobre w .
3. Cinta 3: mantiene el estado actual de M .

Definición 1.3 (Lenguaje Accept).

$$ACCEPT = \{\langle M, w \rangle \mid M \text{ es MT, } w \in \{0,1\}^* \text{ y } M \text{ acepta } w\}.$$

Proposición 1.1. $L(U) = ACCEPT$, por lo tanto $ACCEPT \in \mathcal{RE}$ (recursivamente enumerable).

2. Clase 7 – Problemas Indecidibles

2.1. Diagonal y el argumento de diagonalización

Definición 2.1 (Diagonal).

$$\text{Diagonal} = \{\langle M \rangle \mid M \text{ es MT y } M \text{ no acepta } \langle M \rangle\}.$$

Teorema 2.1. *Diagonal no es recursivamente enumerable ($\notin \mathcal{RE}$) y, por lo tanto, es indecidible ($\notin \mathcal{R}$).*

Demostración (diagonalización). Se supone que existe una MT D tal que $L(D) = \text{Diagonal}$. Ejecutamos D sobre $\langle D \rangle$:

- Si D acepta $\langle D \rangle \implies \langle D \rangle \in \text{Diagonal} \implies D$ no acepta $\langle D \rangle$. **Contradicción.**
- Si D no acepta $\langle D \rangle \implies \langle D \rangle \notin \text{Diagonal} \implies D$ acepta $\langle D \rangle$. **Contradicción.**

Por lo tanto, no puede existir tal MT D . □

Interpretación tabular: Se construye tabla T donde las filas son MTs codificables $M_1, M_2, \dots$ y las columnas son codificaciones $\langle M_1 \rangle, \langle M_2 \rangle, \dots$, con $T[M_i, \langle M_j \rangle] = 1 \iff M_i$ acepta $\langle M_j \rangle$. La fila de Diagonal es el **complemento de la diagonal** de la tabla; como Diagonal difiere de toda fila, no aparece en la tabla $\implies$ no es RE.

2.2. Accept es indecidible

Teorema 2.2. *ACCEPT es indecidible.*

Demostración. Se supone que existe una MT A que decide ACCEPT. Se construye D :

D sobre entrada z :

1. Verificar que $z = \langle M \rangle$ para alguna MT M . Si no, rechazar.
2. Ejecutar A sobre $\langle M, \langle M \rangle \rangle$.
3. Si A acepta, rechazar.
4. Si A rechaza, aceptar.

Entonces $L(D) = \text{Diagonal}$. Pero A decide ACCEPT $\implies D$ decide Diagonal. Contradicción (Diagonal $\notin \mathcal{RE}$). □

Corolario 2.1. $\overline{\text{ACCEPT}}$ (el complemento de Accept) **no es RE**.

Estructura típica para probar indecidibilidad

1. Suponer que existe una MT que decide el lenguaje objetivo.
2. Construir, a partir de ella, una MT que decida ACCEPT (o Diagonal).
3. Concluir contradicción.

3. Clase 8 – Reducciones e Indecidibilidad

3.1. El Problema de la Parada (Halting)

Definición 3.1 (Halting).

$$HALTING = \{\langle M, w \rangle \mid M \text{ es MT, } w \in \{0, 1\}^* \text{ y } M \text{ se detiene con } w\}.$$

Proposición 3.1. $HALTING \in \mathcal{RE}$.

Teorema 3.1. $HALTING$ es indecible ($\notin \mathcal{R}$).

Demostración. Se supone H que decide $HALTING$. Para cada MT M , se define S_M :

S_M sobre entrada w :

1. Ejecutar M sobre w .
2. Si M acepta w , aceptar.
3. Si M rechaza w , quedarse en loop infinito.

Se cumple: $\langle M, w \rangle \in ACCEPT \iff \langle S_M, w \rangle \in HALTING$. Se define A que decide $ACCEPT$ usando H . Contradicción. $\square$

3.2. $Halting_\varepsilon$

Definición 3.2. $HALTING_\varepsilon = \{\langle M \rangle \mid M \text{ es MT y } M \text{ se detiene con } \varepsilon\}$.

$HALTING_\varepsilon \in \mathcal{RE}$ e indecible. Se demuestra $HALTING \leq_m HALTING_\varepsilon$ usando $B_{M,w}$ que borra su entrada, escribe w y ejecuta M .

Corolario 3.1. $\overline{HALTING}$ y $\overline{HALTING_\varepsilon}$ no son RE.

3.3. Reducciones Mucho a Uno ($\leq_m$)

Definición 3.3 (Reducción). Sean $K \subseteq \Sigma_1^*$ y $L \subseteq \Sigma_2^*$. Una **reducción** de K a L es una función computable $f: \Sigma_1^* \rightarrow \Sigma_2^*$ tal que:

$$\forall w \in \Sigma_1^* : w \in K \iff f(w) \in L.$$

Se denota $K \leq_m L$.

Propiedades Fundamentales de $\leq_m$

1. **Propiedad 1:** $K \leq_m L$ y $L \in \mathcal{R} \implies K \in \mathcal{R}$.
Contrapositivo: $K \leq_m L$ y $K \notin \mathcal{R} \implies L \notin \mathcal{R}$.
 (L es “al menos tan difícil” como K .)
2. **Propiedad 2:** $K \leq_m L$ y $L \in \mathcal{RE} \implies K \in \mathcal{RE}$.
Contrapositivo: $K \leq_m L$ y $K \notin \mathcal{RE} \implies L \notin \mathcal{RE}$.
3. **Transitividad:** $K \leq_m L$ y $L \leq_m J \implies K \leq_m J$.
4. **Complemento:** $K \leq_m L \implies \overline{K} \leq_m \overline{L}$ (la misma f sirve).

3.4. Problemas Clásicos

Non-Empty

$\text{Non-Empty} = \{\langle M \rangle \mid M \text{ es MT y } L(M) \neq \emptyset\}.$

$\in \mathcal{RE}$. Indecidible. $\text{ACCEPT} \leq_m \text{Non-Empty}$ mediante $B_{M,w}$ (borra x , escribe w , ejecuta M):

- $M \text{ acepta } w \implies L(B_{M,w}) = \{0,1\}^* \in \text{Non-Empty}.$
- $M \text{ no acepta } w \implies L(B_{M,w}) = \emptyset \notin \text{Non-Empty}.$

Corolario 3.2. $\overline{\text{Non-Empty}} \notin \mathcal{RE}$ (pues $\overline{\text{ACCEPT}} \leq_m \overline{\text{Non-Empty}}$).

All

$\text{All} = \{\langle M \rangle \mid M \text{ es MT y } L(M) = \{0,1\}^*\}.$

$\notin \mathcal{RE}$ (por lo tanto indecible). Dos reducciones desde ACCEPT :

- $\text{ACCEPT} \leq_m \text{All}$: misma $B_{M,w}$. Si M acepta w , $L = \{0,1\}^* \in \text{All}$; si no, $L = \emptyset \notin \text{All}$.
- $\overline{\text{ACCEPT}} \leq_m \text{All}$ (alternativa con $Z_{M,w}$): ejecuta M sobre w por $|x|$ pasos; si M acepta, rechaza; si no, acepta.

Finite

$\text{Finite} = \{\langle M \rangle \mid M \text{ es MT y } L(M) \text{ es finito}\}.$

$\notin \mathcal{RE}$ (indecidible). Reducción: $\overline{\text{ACCEPT}} \leq_m \text{Finite}$ usando $Z_{M,w}$.

- $M \text{ no acepta } w \implies L(Z_{M,w}) = \{0,1\}^* \text{ (infinito)} \notin \text{Finite}.$
- $M \text{ acepta } w \text{ en } k \text{ pasos} \implies Z_{M,w} \text{ rechaza todo } x \text{ con } |x| \geq k \implies L(Z_{M,w}) \text{ es finito} \in \text{Finite}.$

Estrategia general de reducción para ejercicios

1. Identificar un lenguaje conocido indecible (Accept o Halting).
2. Diseñar una función computable f que transforme instancias.
3. Demostrar la equivalencia: $\langle M, w \rangle \in \text{Accept} \iff f(\langle M, w \rangle) \in L_{\text{target}}.$
4. Concluir que L_{target} es indecible (o no RE, según el caso).
5. Para demostrar no RE, usar $\leq_m$ desde $\overline{\text{ACCEPT}}$.

4. Clase 9 – Más Indecidibilidad: PCP

4.1. Problema de Correspondencia de Post (PCP)

Definición 4.1. ■ Un **dominó** sobre alfabeto Σ es un par (x, y) , $x, y \in \Sigma^*$, escrito $\begin{bmatrix} x \\ y \end{bmatrix}$.

- Una **solución** para un conjunto P de dominós es una secuencia finita $\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \dots, \begin{bmatrix} x_k \\ y_k \end{bmatrix}$ ($k \geq 1$) de P tal que $x_1 \cdots x_k = y_1 \cdots y_k$.
- $PCP = \{\langle P \rangle \mid P \text{ es un conjunto de dominós que tiene solución}\}$.

4.2. PCP Modificado (MPCP)

Definición 4.2. $MPCP = \{\langle P, d \rangle \mid P \text{ conjunto de dominós, } d \in P, \text{ y } P \text{ tiene solución que parte con } d\}$.

Teorema 4.1. $PCP \in \mathcal{RE}$ pero es **indecidable**.

Demostración en dos pasos: $ACCEPT \leq_m MPCP$ y $MPCP \leq_m PCP$.

4.3. $ACCEPT \leq_m MPCP$ (Construcción)

Dada MT M y $w = a_1 \cdots a_n$, se construye $P_{M,w}$ y dominó inicial $d_{M,w}$ tal que las soluciones representan **ejecuciones de aceptación** de M sobre w , escritas como $\#C_0\#C_1\#\cdots C_m\#$.

Las partes de la construcción son:

1. **Dominó inicial:** $\begin{bmatrix} \# \\ \# \vdash q_0 a_1 \cdots a_n \# \end{bmatrix}$
2. **Transición derecha** $\delta(q, a) = (r, b, \triangleright)$: agregar $\begin{bmatrix} qa \\ br \end{bmatrix}$.
3. **Transición izquierda** $\delta(q, a) = (r, b, \triangleleft)$: para todo $c \in \Gamma$, agregar $\begin{bmatrix} cqa \\ rcb \end{bmatrix}$.
4. **Letras no leídas:** $\begin{bmatrix} a \\ a \end{bmatrix}$ para cada $a \in \Gamma$.
5. **Delimitador:** $\begin{bmatrix} \# \\ \# \end{bmatrix}$ y $\begin{bmatrix} \# \\ B\# \end{bmatrix}$.
6. **Estado final:** $\begin{bmatrix} aq_f \\ q_f \end{bmatrix}$ y $\begin{bmatrix} q_f a \\ q_f \end{bmatrix}$ para cada $a \in \Gamma$.
7. **Cierre:** $\begin{bmatrix} q_f \# \# \\ \# \end{bmatrix}$.

La parte inferior siempre está “adelantada” respecto a la superior, forzando la simulación paso a paso. Solo se igualan al eliminar el estado final.

4.4. $MPCP \leq_m PCP$

Dado P con dominó inicial $\begin{bmatrix} t_1 \\ b_1 \end{bmatrix}$, se define P' insertando $*$ entre símbolos con reglas especiales para forzar que la solución comience con el dominó inicial:

$$P' = \left\{ \begin{bmatrix} *t_1 \\ *b_1* \end{bmatrix}, \begin{bmatrix} *t_1 \\ b_1* \end{bmatrix}, \begin{bmatrix} *t_2 \\ b_2* \end{bmatrix}, \dots, \begin{bmatrix} *t_m \\ b_m* \end{bmatrix}, \begin{bmatrix} *\# \\ \# \end{bmatrix} \right\}$$

(adaptado del esquema visto en clase).

Parte II

Unidad II: Lógica de Primer Orden (LPO)

5. Clase 10 – LPO: Sintaxis y Semántica

5.1. Vocabulario $\mathcal{L}$

Definición 5.1 (Vocabulario). *Un vocabulario $\mathcal{L} = \{C, F, R\}$ donde:*

- $C = \{c_1, c_2, \dots\}$: *símbolos de constantes.*
- $F = \{f_1, f_2, \dots\}$: *símbolos de funciones, cada uno con **aridad** $n_i > 0$.*
- $R = \{R_1, R_2, \dots\}$: *símbolos de relaciones, cada uno con **aridad** $m_i \geq 0$.*

Ejemplos: $\mathcal{L} = \{0, 1, \text{suc}, +, \cdot, <\}$ (naturales); $\mathcal{L} = \{E\}$ (grafos); $\mathcal{L} = \{\in\}$ (conjuntos).

5.2. Sintaxis: $\mathcal{L}$ -términos

Sea $V = \{x, y, \dots\}$ un conjunto infinito de variables.

Definición 5.2 ($\mathcal{L}$ -términos). ▪ *Toda constante $c \in \mathcal{L}$ es un $\mathcal{L}$ -término.*

- *Toda variable $x \in V$ es un $\mathcal{L}$ -término.*
- *Si $t_1, \dots, t_n$ son $\mathcal{L}$ -términos y $f \in \mathcal{L}$ es función n -aria, entonces $f(t_1, \dots, t_n)$ es un $\mathcal{L}$ -término.*

5.3. Sintaxis: $\mathcal{L}$ -fórmulas

Definición 5.3 ($\mathcal{L}$ -fórmulas). ▪ **Atómicas:** $t_1 = t_2$ y $R(t_1, \dots, t_m)$, con t_i $\mathcal{L}$ -términos y $R \in \mathcal{L}$ relación m -aria.

- **Compuestas:** $(\neg\varphi), (\varphi \vee \psi), (\varphi \wedge \psi), (\varphi \rightarrow \psi), (\varphi \leftrightarrow \psi)$.
- **Cuantificadas:** $(\exists x \varphi), (\forall x \varphi)$.

5.4. Semántica: $\mathcal{L}$ -estructuras

Definición 5.4 ($\mathcal{L}$ -estructura). *Una $\mathcal{L}$ -estructura $\mathcal{A}$ consiste en:*

- *Un dominio $A \neq \emptyset$.*
- *Por cada $c \in \mathcal{L}$: un elemento $c^{\mathcal{A}} \in A$.*
- *Por cada $f \in \mathcal{L}$ n -aria: una función $f^{\mathcal{A}} : A^n \rightarrow A$.*
- *Por cada $R \in \mathcal{L}$ m -aria: una relación $R^{\mathcal{A}} \subseteq A^m$.*

5.5. Variables libres y ligadas

- $V(t)$: conjunto de variables de un término t .
- $V(\varphi)$: conjunto de variables de una fórmula φ .
- $VL(\varphi)$: variables libres de φ (las no cuantificadas):

$$VL(t_1 = t_2) = V(t_1) \cup V(t_2)$$

$$VL(\neg\psi) = VL(\psi)$$

$$VL(\psi_1 \star \psi_2) = VL(\psi_1) \cup VL(\psi_2)$$

$$VL(\exists x \psi) = VL(\forall x \psi) = VL(\psi) \setminus \{x\}$$

- $\varphi(x_1, \dots, x_k)$ indica que $VL(\varphi) = \{x_1, \dots, x_k\}$.
- **Oración:** fórmula sin variables libres ($VL(\varphi) = \emptyset$).

5.6. Asignaciones y satisfacción $\models$

Definición 5.5 (Asignación). Una asignación es una función $\sigma : V \rightarrow A$. Su extensión $\hat{\sigma}$ a $\mathcal{L}$ -términos se define:

$$\begin{aligned}\hat{\sigma}(c) &= c^{\mathcal{A}} \\ \hat{\sigma}(x) &= \sigma(x) \\ \hat{\sigma}(f(t_1, \dots, t_n)) &= f^{\mathcal{A}}(\hat{\sigma}(t_1), \dots, \hat{\sigma}(t_n))\end{aligned}$$

Definición 5.6 (Satisfacción $(\mathcal{A}, \sigma) \models \varphi$). Por inducción sobre φ :

$$\begin{aligned}(\mathcal{A}, \sigma) \models t_1 = t_2 &\iff \hat{\sigma}(t_1) = \hat{\sigma}(t_2) \\ (\mathcal{A}, \sigma) \models R(t_1, \dots, t_n) &\iff (\hat{\sigma}(t_1), \dots, \hat{\sigma}(t_n)) \in R^{\mathcal{A}} \\ (\mathcal{A}, \sigma) \models \neg \varphi &\iff (\mathcal{A}, \sigma) \not\models \varphi \\ (\mathcal{A}, \sigma) \models \varphi \vee \psi &\iff (\mathcal{A}, \sigma) \models \varphi \text{ o } (\mathcal{A}, \sigma) \models \psi \\ (\mathcal{A}, \sigma) \models \varphi \wedge \psi &\iff (\mathcal{A}, \sigma) \models \varphi \text{ y } (\mathcal{A}, \sigma) \models \psi \\ (\mathcal{A}, \sigma) \models \exists x \varphi &\iff \text{existe } a \in A : (\mathcal{A}, \sigma[x/a]) \models \varphi \\ (\mathcal{A}, \sigma) \models \forall x \varphi &\iff \text{para todo } a \in A : (\mathcal{A}, \sigma[x/a]) \models \varphi\end{aligned}$$

$$\text{donde } \sigma[x/a](y) = \begin{cases} a & \text{si } y = x \\ \sigma(y) & \text{si } y \neq x \end{cases}.$$

Notación con tuplas: $\mathcal{A} \models \varphi(a_1, \dots, a_k)$ significa $(\mathcal{A}, \sigma) \models \varphi$ con $\sigma(x_i) = a_i$.

5.7. Equivalencia lógica

Definición 5.7. $\varphi \equiv \psi$ si para toda $\mathcal{L}$ -estructura $\mathcal{A}$ y asignación σ : $(\mathcal{A}, \sigma) \models \varphi \iff (\mathcal{A}, \sigma) \models \psi$.

Equivalencias importantes:

$$\begin{aligned}\neg(\exists x \varphi) &\equiv \forall x(\neg \varphi) & \neg(\forall x \varphi) &\equiv \exists x(\neg \varphi) \\ \exists x(\varphi \vee \psi) &\equiv (\exists x \varphi) \vee (\exists x \psi) & \forall x(\varphi \wedge \psi) &\equiv (\forall x \varphi) \wedge (\forall x \psi)\end{aligned}$$

Cuidado: $\exists x(\varphi \wedge \psi) \not\equiv (\exists x \varphi) \wedge (\exists x \psi)$ en general.

5.8. Consecuencia lógica, satisfacibilidad, validez

Definición 5.8. ▪ $\Sigma \models \varphi$ (φ es consecuencia lógica de Σ): para toda $\mathcal{A}, \sigma$, si $(\mathcal{A}, \sigma) \models \Sigma$ entonces $(\mathcal{A}, \sigma) \models \varphi$.

- φ es **satisfacible** si $\exists \mathcal{A}, \sigma : (\mathcal{A}, \sigma) \models \varphi$.
- φ es **válida** ($\models \varphi$) si $\forall \mathcal{A}, \sigma : (\mathcal{A}, \sigma) \models \varphi$.
- Σ es **satisfacible (consistente)** si $\exists \mathcal{A}, \sigma : (\mathcal{A}, \sigma) \models \Sigma$.
- Σ es **inconsistente** si no tiene modelo.

Propiedades clave

- φ válida $\iff \neg \varphi$ no es satisfacible.
- $\Sigma \models \varphi \iff \Sigma \cup \{\neg \varphi\}$ es inconsistente.

- Si Σ es inconsistente, entonces $\Sigma \models \varphi$ para todo φ .
- φ válida $\iff \emptyset \models \varphi$.

5.9. Problemas de decisión en LPO

- $\text{SAT} = \{\langle \varphi \rangle \mid \varphi \text{ es fórmula satisfacible}\}$ – indecidible (Church-Turing).
- $\text{VAL} = \{\langle \varphi \rangle \mid \varphi \text{ es fórmula válida}\}$ – indecidible. $\varphi \in \text{VAL} \iff \neg \varphi \notin \text{SAT}$.

6. Clase 11 – Definibilidad sobre una Estructura e Isomorfismos

6.1. Definibilidad sobre una estructura

Definición 6.1 (Relación definible). Sea $\mathcal{A}$ una $\mathcal{L}$ -estructura con dominio A . Una relación $S \subseteq A^k$ ($k \geq 1$) es **definible en $\mathcal{A}$** si existe una $\mathcal{L}$ -fórmula $\varphi(x_1, \dots, x_k)$ tal que:

$$S = \{(a_1, \dots, a_k) \in A^k \mid \mathcal{A} \models \varphi(a_1, \dots, a_k)\}.$$

Ejemplo 6.1 (En $(\mathbb{N}, +)$).

$$\text{Sucesor: } \varphi_{\text{suc}}(x, y) = \varphi$$

$$\begin{aligned} & (x, y) \wedge \neg \exists z (\varphi \\ & (x, z) \wedge \varphi \\ & (z, y)) \end{aligned}$$

$$\text{Uno: } \varphi_1(x) = \forall y (\varphi_0(y) \rightarrow \varphi_{\text{suc}}(y, x))$$

$$\text{Cero: } \varphi_0(x) = (x + x = x) \text{ Menor que: } \varphi$$

$$(x, y) = \exists z (x + z = y \wedge \neg x = y)$$

$$\text{Sucesor: } \varphi_{\text{suc}}(x, y) = \varphi$$

$$\begin{aligned} & (x, y) \wedge \neg \exists z (\varphi \\ & (x, z) \wedge \varphi \\ & (z, y)) \end{aligned}$$

$$\text{Uno: } \varphi_1(x) = \forall y (\varphi_0(y) \rightarrow \varphi_{\text{suc}}(y, x))$$

Ejemplo 6.2 (En $(\mathbb{R}, +, \cdot)$).

$$\text{Cero: } \varphi_0(x) = (x + x = x)$$

$$\text{No negativos: } \varphi_{\geq 0}(x) = \exists y (y \cdot y = x)$$

$$\text{Menor o igual: } \varphi_{\leq}(x, y) = \exists z (\varphi_{\geq 0}(z) \wedge x + z = y)$$

6.2. Isomorfismos

Definición 6.2 (Isomorfismo). Sean $\mathcal{A}$ y $\mathcal{B}$ dos $\mathcal{L}$ -estructuras con dominios A, B . $\mathcal{A} \cong \mathcal{B}$ si existe una biyección $h : A \rightarrow B$ tal que:

- $h(c^{\mathcal{A}}) = c^{\mathcal{B}}$ para toda constante $c \in \mathcal{L}$.
- $h(f^{\mathcal{A}}(a_1, \dots, a_n)) = f^{\mathcal{B}}(h(a_1), \dots, h(a_n))$ para toda función f n -aria.
- $(a_1, \dots, a_m) \in R^{\mathcal{A}} \iff (h(a_1), \dots, h(a_m)) \in R^{\mathcal{B}}$ para toda relación R m -aria.

h se llama **isomorfismo** de $\mathcal{A}$ a $\mathcal{B}$.

6.3. Teorema de Isomorfismo

Teorema 6.1 (Teorema de Isomorfismo – versión oraciones). Si $\mathcal{A} \cong \mathcal{B}$, entonces para toda $\mathcal{L}$ -oración φ :

$$\mathcal{A} \models \varphi \iff \mathcal{B} \models \varphi.$$

LPO no puede distinguir estructuras isomorfas.

Teorema 6.2 (Teorema de Isomorfismo – versión general). Si $h : \mathcal{A} \rightarrow \mathcal{B}$ es isomorfismo, para toda $\mathcal{L}$ -fórmula $\varphi(x_1, \dots, x_k)$ y $(a_1, \dots, a_k) \in A^k$:

$$\mathcal{A} \models \varphi(a_1, \dots, a_k) \iff \mathcal{B} \models \varphi(h(a_1), \dots, h(a_k)).$$

6.4. Isomorfismos y no definibilidad

Proposición 6.1 (Clave para demostrar no definibilidad). Sea $\mathcal{A}$ una $\mathcal{L}$ -estructura y $S \subseteq A^k$. Si existe un isomorfismo $h : \mathcal{A} \rightarrow \mathcal{A}$ y $(a_1, \dots, a_k) \in S$ con $(h(a_1), \dots, h(a_k)) \notin S$, entonces S **no es definible** en $\mathcal{A}$.

Ejemplo 6.3. La multiplicación M no es definible en $(\mathbb{R}, +)$: $h(x) = 2x$ es isomorfismo de $(\mathbb{R}, +)$ en sí mismo; $(1, 1, 1) \in M$ pero $(2, 2, 2) \notin M$. Por la proposición, M no es definible.

Observación 6.1. En $(\mathbb{N}, +)$ el único isomorfismo en sí mismo es la identidad, por lo que este método no sirve para demostrar que $\times$ no es definible en $(\mathbb{N}, +)$.

Técnica para NO definibilidad sobre una estructura

Para demostrar que $S \subseteq A^k$ NO es definible en $\mathcal{A}$:

1. Encontrar un isomorfismo $h : \mathcal{A} \rightarrow \mathcal{A}$ que no respeta S .
2. Exhibir $(a_1, \dots, a_k) \in S$ tal que $(h(a_1), \dots, h(a_k)) \notin S$.
3. Por la proposición anterior, S no es definible.

7. Clase 12 – Definibilidad de Clases de Estructuras y Compacidad

7.1. Propiedades sobre $\mathcal{L}$ -estructuras

Definición 7.1. ■ $Struct[\mathcal{L}] = \{\mathcal{A} \mid \mathcal{A} \text{ es una } \mathcal{L}\text{-estructura}\}.$

- Una **propiedad** P es un subconjunto $P \subseteq Struct[\mathcal{L}]$.
- P es **definible en LPO** si existe una $\mathcal{L}$ -oración φ tal que: $\forall \mathcal{A} \in Struct[\mathcal{L}] : \mathcal{A} \models \varphi \iff \mathcal{A} \in P.$

Ejemplo 7.1 ($\mathcal{L} = \{E\}$, grafos dirigidos). ■ “Tiene al menos 2 elementos”: $\varphi = \exists x \exists y \neg(x = y).$

- “Tiene un ciclo dirigido de largo 3”: $\varphi = \exists x \exists y \exists z (E(x, y) \wedge E(y, z) \wedge E(z, x)).$
- “Grado de salida ≤ 2 ”: $\varphi = \neg \exists x \exists y_1 \exists y_2 \exists y_3 (\neg y_1 = y_2 \wedge \neg y_2 = y_3 \wedge \neg y_1 = y_3 \wedge E(x, y_1) \wedge E(x, y_2) \wedge E(x, y_3)).$

7.2. Propiedades cerradas bajo isomorfismos

Definición 7.2. P es **cerrada bajo isomorfismos** si para todo $\mathcal{A} \cong \mathcal{B} : \mathcal{A} \in P \iff \mathcal{B} \in P.$

Proposición 7.1. Si P es definible en LPO $\implies P$ es cerrada bajo isomorfismos.

Corolario 7.1 (Contrapositivo). Si P no es cerrada bajo isomorfismos $\implies P$ no es definible en LPO.

7.3. Teorema de Compacidad

Teorema 7.1 (Teorema de Compacidad). Sea Σ un conjunto de $\mathcal{L}$ -oraciones. Entonces:

$$\Sigma \text{ es satisfacible} \iff \Sigma \text{ es finitamente satisfacible.}$$

(donde “finitamente satisfacible” significa que todo subconjunto finito de Σ es satisfacible.)

Este teorema es la herramienta fundamental para demostrar **no definibilidad** de propiedades en LPO.

7.4. Estrategia de compacidad para no definibilidad

Para demostrar que P NO es definible en LPO

1. **Suponer** que existe $\mathcal{L}$ -oración φ que define P .
2. **Construir** $\Sigma = \{\varphi\} \cup \Gamma$ donde $\Gamma = \{\gamma_n \mid n \in \mathbb{N}\}$ es un conjunto infinito de oraciones.
3. **Demostrar** que Σ **no es satisfacible** (φ y Γ “dicen lo contrario”).
4. **Demostrar** que Σ es **finitamente satisfacible** (todo subconjunto finito $\Delta \subseteq \Sigma$ tiene modelo).
5. Por compacidad, Σ sería satisfacible. **Contradicción.**

7.5. Ejemplos clásicos de no definibilidad por compacidad

7.5.1. P_{fin} = estructuras finitas

Supongamos que φ define P_{fin} . Para cada $n \geq 2$, sea

$$\lambda_n = \exists x_1 \cdots \exists x_n \left(\bigwedge_{1 \leq i < j \leq n} \neg x_i = x_j \right)$$

(“tiene al menos n elementos”).

Sea $\Sigma = \{\varphi\} \cup \{\lambda_n \mid n \geq 2\}$.

- Σ no es satisfacible: φ exige finitud, $\{\lambda_n\}$ exige infinitud.
- Σ es finitamente satisfacible: dado $\Delta \subseteq \Sigma$ finito, sea $k = \max\{n \mid \lambda_n \in \Delta\}$; una estructura de k elementos satisface Δ .

Por compacidad, Σ es satisfacible. **Contradicción.**

7.5.2. $P_{\text{path}} = \text{existe camino dirigido de } c \text{ a } d \text{ en } \mathcal{A}$

$\mathcal{L} = \{c, d, E\}$. Supongamos que φ define P_{path} . Para $n \geq 0$:

$$\begin{aligned}\rho_0 &= (\neg c = d) \\ \rho_n &= \neg(\exists x_1 \cdots \exists x_{n-1} (E(c, x_1) \wedge E(x_1, x_2) \wedge \cdots \wedge E(x_{n-1}, d))) \quad (n \geq 1)\end{aligned}$$

ρ_n dice “no hay camino de largo n de c a d ”.

Sea $\Sigma = \{\varphi\} \cup \{\rho_n \mid n \geq 0\}$.

- Σ no es satisfacible.
- Σ es finitamente satisfacible: dado $\Delta \subseteq \Sigma$ finito, sea $k = \max\{n \mid \rho_n \in \Delta\}$; un camino de largo $k + 1$ satisface Δ .

Contradicción por compacidad.

7.5.3. $P_{\text{cycle}} = \text{existe un ciclo dirigido finito}$

$\mathcal{L} = \{E\}$. Similar: se combina la existencia de ciclo con oraciones ρ_n que fuerzan que no hay ciclos de ningún largo fijo n . El subconjunto finito con $\rho_1, \dots, \rho_k$ se satisface con una estructura que tiene un ciclo de largo $k + 1$.

7.5.4. $P_{\text{fin-degree}} = \text{existe elemento con grado de salida finito}$

$\mathcal{L} = \{E\}$. Similar: Σ combina la propiedad con oraciones que fuerzan que todo elemento tiene al menos n vecinos de salida. Un subconjunto finito se satisface con una estructura donde todos tienen grado de salida finito pero arbitrariamente grande.

Proposición 7.2 (Propiedad importante). *Si P es definible en LPO, entonces $\overline{P}$ (el complemento) también es definible (basta negar la oración). Por lo tanto, P_{inf} (estructuras con dominio infinito) tampoco es definible en LPO.*

Parte III

Herramientas y Técnicas para Ejercicios

8. Guía de Computabilidad – Técnicas Esenciales

8.1. Diseño de MTs (Problemas 1–3 de la guía)

- **MT determinista 1 cinta:** Describir $Q, \Sigma, \Gamma, q_0, q_f, \delta$ formalmente. Usar notación de módulos (subrutinas) para simplificar.
- **Palíndromos:** Marcar extremos, comparar y avanzar.
- **Operaciones aritméticas (unario):** sumas ($0^i \# 0^j \# 0^k, i + j = k$), divisibilidad (n divide m).
- **Funciones computables:** MT que transforma entrada w en salida $h(w)$ sobre la cinta. Debe terminar siempre con $\vdash q_f h(w)$.

8.2. Modelos extendidos de MT (Problemas 4–6)

Todo modelo razonable de MT es equivalente al modelo determinista básico (Tesis de Church-Turing):

- **Múltiples estados iniciales:** Intentar cada estado inicial.
- **Estados iniciales universales:** Ejecutar desde cada $q \in I$ y aceptar si todas aceptan.
- **1-MTUA (universalmente alternante):** Simular todas las ejecuciones; aceptar si todas son finitas y aceptan.

8.3. Lenguajes decidibles y RE (Problemas 7–10)

- **Lenguaje finito:** Siempre decidable.
- $\mathcal{R}$ cerrado bajo $\cup$ e $\cap$.
- $\mathcal{RE}$ cerrado bajo $\cup$ e $\cap$.
- **Homomorfismo:** $h(uv) = h(u)h(v)$, $h(\varepsilon) = \varepsilon$. Todo homomorfismo es computable.

8.4. Reducciones – Técnicas (Problemas 16–20)

Patrones comunes de reducción:

- $B_{M,w}$ (**borrar y ejecutar**): Borra entrada x , escribe w , ejecuta M sobre w . Útil para reducir ACCEPT a lenguajes sobre lenguaje aceptado.
- S_M (**aceptación a parada**): Ejecuta M ; si acepta, acepta; si rechaza, loop infinito. Convierte aceptación en detención.
- $Z_{M,w}$ (**ejecución por pasos**): Ejecuta M sobre w por $|x|$ pasos. Invierte aceptación/rechazo. Útil para reducir ACCEPT y mostrar no RE.

Esquema para indecidibilidad vía reducción

1. Elegir lenguaje indecidible conocido (ACCEPT o HALTING).
2. Definir f computable.
3. Demostrar: $\langle M, w \rangle \in \text{ACCEPT} \iff f(\langle M, w \rangle) \in L$.

4. Concluir L indecidible.

Esquema para no RE vía reducción

1. Elegir lenguaje no RE conocido ($\overline{\text{ACCEPT}}$ o $\overline{\text{HALTING}}$).
2. Misma estrategia de reducción.
3. Concluir $L \notin \mathcal{RE}$ (por Propiedad 2 contrapositiva de $\leq_m$).

9. Guía de Lógica – Técnicas Esenciales

9.1. Demostraciones con $\models$ y satisfacibilidad (Problemas 1–2)

- $\Sigma \models \varphi \iff \Sigma \cup \{\neg\varphi\}$ inconsistente: herramienta fundamental para reducir consecuencia lógica a inconsistencia.
- Para reducir $\text{Equiv} \leq_m \text{Val}$: construir $\alpha_{\varphi,\psi}$ tal que $\varphi \equiv \psi \iff \models \alpha_{\varphi,\psi}$. Por ejemplo, $\alpha_{\varphi,\psi} = (\varphi \leftrightarrow \psi)$.

9.2. Isomorfismos – Demostrar que dos estructuras NO son isomorfas (Problemas 3–6)

Buscar una propiedad expresable en LPO que distinga las estructuras:

- $(\mathbb{N}, <) \not\cong (\mathbb{Q}, <)$: $\mathbb{Q}$ es denso, $\mathbb{N}$ no.
- $(\mathbb{N}, +) \not\cong (\mathbb{Z}, +)$: $\mathbb{Z}$ tiene inversos aditivos, $\mathbb{N}$ no.
- $(\mathbb{Z}, +) \not\cong (\mathbb{Q}, +)$: $\mathbb{Q}$ es divisible ($\forall x \exists y y + y = x$), $\mathbb{Z}$ no.
- $(\mathbb{N}, \cdot) \not\cong (\mathbb{Z}, \cdot)$: diferentes propiedades (elementos neutros, absorción).

9.3. Definibilidad sobre una estructura (Problemas 7–14)

Para demostrar que S SÍ es definible

Construir una $\mathcal{L}$ -fórmula φ que capture exactamente S .

- $S = \{0\}$ en $(\mathbb{N}, \cdot)$: $\varphi(x) = \forall y (x \cdot y = x)$.
- $S = \{\text{primos}\}$ en $(\mathbb{N}, \cdot)$: p es primo $\iff p \geq 2$ y $\forall y, z (p = y \cdot z \rightarrow y = 1 \vee z = 1)$.
- “ n divide m ” en $(\mathbb{N}, \cdot)$: $\varphi(x, y) = \exists z (y = x \cdot z)$.

Para demostrar que S NO es definible

Encontrar un isomorfismo $h : \mathcal{A} \rightarrow \mathcal{A}$ que no respeta S .

- Multiplicación no es definible en $(\mathbb{R}, +)$: $h(x) = 2x$.
- $<$ no es definible en $(\mathbb{R}, +)$: $h(x) = -x$.
- $\mathbb{N}$ no es definible en $(\mathbb{R}, <)$: usar isomorfismo que “estira” $\mathbb{R}$.
- $+$ no es definible en $(\mathbb{R}, \cdot)$: $h(x) = x^3$ (respeta $\cdot$, no $+$).

Cerradura de relaciones definibles (Problemas 15–16): Si S y T son definibles sobre $\mathcal{A}$, entonces $\bar{S}$, $S \cup T$, $S \cap T$, y $S \circ T$ también lo son.

9.4. Definibilidad de propiedades/clases (Problemas 17–24)

Para demostrar que P SÍ es definible

Escribir una $\mathcal{L}$ -oración que capture P . Ejercitar con: órdenes parciales, totales, lattices, máximos, maximales, densidad, elementos neutros, inversos, conmutatividad, asociatividad, distributividad, inyectividad, sobreyectividad, monotonía, puntos fijos.

Para demostrar que P NO es definible

Dos métodos:

1. Demostrar que P no es cerrada bajo isomorfismos.
2. Usar el **teorema de compacidad** (método principal):
 - Asumir φ define P .
 - Construir $\Gamma = \{\gamma_n \mid n \in \mathbb{N}\}$ tal que cada γ_n fuerza una propiedad contraria a P .
 - $\Sigma = \{\varphi\} \cup \Gamma$ no es satisfacible.
 - Todo $\Delta \subseteq \Sigma$ finito es satisfacible (construir estructura que satisface φ y γ_n con $n \leq k$).
 - Por compacidad, contradicción.

Problemas típicos con compacidad en la guía:

- P_{fin} y P_{inf} no son definibles (Problema 22).
- P_{cycle} (existencia de ciclo finito) no es definible (Problema 23).
- $P_{\text{fin-degree}}$ (existencia de grado de salida finito) no es definible (Problema 24).

Parte IV

Tablas Resumen

10. Jerarquía de Lenguajes en Computabilidad

Lenguaje	$\in \mathcal{R}$?	$\in \mathcal{RE}$?	$\in \text{co-}\mathcal{RE}$?	Notas
ACCEPT	NO	SI	NO	$\overline{\text{ACCEPT}} \notin \mathcal{RE}$
HALTING	NO	SI	NO	$\text{HALTING} \in \mathcal{RE}$, indecible
$\text{HALTING}_\varepsilon$	NO	SI	NO	$\text{HALTING} \leq_m \text{HALTING}_\varepsilon$
Diagonal	NO	NO	NO	Ni RE ni co-RE
Non-Empty	NO	SI	NO	$\text{ACCEPT} \leq_m \text{Non-Empty}$
All	NO	NO	NO	$\overline{\text{ACCEPT}} \leq_m \text{All}$ (no RE)
Finite	NO	NO	NO	$\overline{\text{ACCEPT}} \leq_m \text{Finite}$
PCP	NO	SI	NO	$\text{ACCEPT} \leq_m \text{MPCP} \leq_m \text{PCP}$

Nota: $\mathcal{R} = \mathcal{RE} \cap \text{co-}\mathcal{RE}$ (un lenguaje es decible sii tanto él como su complemento son RE).

11. Herramientas de LPO para Demostraciones

Concepto	Para demostrar SÍ	Para demostrar NO
Definibilidad de S sobre $\mathcal{A}$	Escribir φ que define S	Isomorfismo $h : \mathcal{A} \rightarrow \mathcal{A}$ que no respeta S
Propiedad P definible (clase)	Escribir oración φ que define P	Compacidad / P no cerrada bajo $\cong$
$\mathcal{A} \cong \mathcal{B}$ (isomorfismo)	Construir biyección h que preserva estructura	Propiedad LPO que distingue
$\Sigma \models \varphi$ (consecuencia)	Definición directa $\forall \mathcal{A}, \sigma$	$\Sigma \cup \{\neg\varphi\}$ inconsistente
φ satisfacible	Construir $\mathcal{A}$ y σ	Reducción a problema conocido